4.5 Casting

LLVM has two pointer/integer casting instructions: **ptrtoint** and **inttoptr**. The semantics of casting a logical pointer to an integer is given in PTRTOINT-LOGICAL. Function $\mathsf{cast2int}_M(l, o, s)$ converts $\mathsf{Log}(l, o, s)$ to the integer $P(s)_0 + o$ based on block l. If the operation $P(s)_0 + o$ overflows, it wraps around. This can happen if the pointer is out of bounds. Instruction '**ptrtoint** $\mathsf{Phy}(o, s, I, cid)$' yields o. If the size of the destination type **i**sz is larger than the pointer width, the result is zero-extended. If it is smaller, the most significant bits are truncated.

Casting from an integer to a pointer returns a physical pointer with no provenance information (INTTOPTR). No check is done on whether the pointer refers to a valid location or not. This avoids a dependency on the memory state, and thus allows code motion.

The NULL pointer used in, e.g., C is defined as '**inttoptr** 0' because C programs use '`(void*)0`' as the null pointer value.

Casting a pointer to a different address space through **addrspacecast** translates the pointer's offset(s) using a target-specific mapping function. If the pointer is logical, it preserves the block id and offset if in bounds, otherwise it yields **poison**. If the pointer is physical, '**ptrtoint** $\mathsf{Phy}(o, s, I, cid)$' updates o as well as the offsets in I.

In our semantics, all casting instructions can be freely moved, removed, or introduced.

4.6 Pointer Comparison

To support pointer optimizations, pointer comparison must be defined as something more elaborate than simply comparing the underlying machine addresses. For example, the following program compares two logical pointers that obviously point to different objects. We would like, therefore, to fold this comparison to false:

```
char *p = malloc(4);
char *q = malloc(4);

char *pp = some expr over p;
char *qq = some expr over q;
if (pp == qq) { /* always  false? */ }
```

If `pp` and `qq` are dereferenceable, i.e., their offsets are in the range $[0, 4)$, the comparison should yield `false` since the resulting machine addresses cannot be the same (since objects `p` and `q` cannot overlap). However, what if `pp == p + 4`, `qq == q`, and the objects `p` and `q` were allocated consecutively (i.e., `p + 4 == q`)? Even though `pp` and `qq` are conceptually very different pointers, their underlying machine addresses are the same. Therefore, if the compiler lowers pointer comparison into a comparison of the respective machine addresses in assembly, the comparison would yield `true`.

In our semantics, we define the comparison `p + n == q` to yield a non-deterministic value, justifying both the lowering to machine address comparison, and the desired optimization. This way the compiler can always fold comparisons between different objects to `false` without having to prove that they cannot have the same machine address.

This optimization corresponds to language in the C++ standard that allows an indeterminate result when comparing non-dereferenceable pointers. In contrast, the C standard inconveniently specifies that the physical values of the pointers must be compared. This implies that `p + n == q` must evaluate to `true` if `p` and `q` were allocated consecutively.

Formally speaking, rule ICMP-PTR-LOGICAL' defines that comparing logical pointers to different blocks can always evaluate to `false`. Furthermore, ICMP-PTR-LOGICAL-NONDET-TRUE states that if the offset of either pointer is not dereferenceable, the comparison can *also* evaluate to `true`.